Chapter 14 worked solutions

Selective State Spaces

This guide covers Exercises 1–4 and 10. It follows the old state and the new write through scalar updates, explains how the step size affects their balance, and traces each input to the final read in a structured sequence matrix. The companion notebook covers Exercises 5–9 and is also available as a PDF.

1. Trace a selective state

Think of the state as a single number that summarizes the inputs so far. At position t, the scalar system keeps a fraction a_t of the old state and adds a fraction $1 - a_t$ of the input at that position. The two terms add to give the new state.

$$x_t = a_t x_{t-1} + (1 - a_t)u_t.$$

Position 1 starts from $x_0 = 0$, so its old-state contribution is zero. The input 4 has write coefficient $1 - 1/2 = 1/2$, which gives $x_1 = 2$. Use this new state as the old state in the second row, and use the result of the second row in the third.

Position t	Input u_t	Old-state contribution $a_t x_{t-1}$	New-input contribution $(1 - a_t)u_t$	Updated state x_t
1	4	$\frac{1}{2}(0) = 0$	$\frac{1}{2}(4) = 2$	$0 + 2 = 2$
2	10	$\frac{9}{10}(2) = \frac{9}{5}$	$\frac{1}{10}(10) = 1$	$\frac{9}{5} + 1 = \frac{14}{5}$
3	2	$\frac{1}{4}(\frac{14}{5}) = \frac{7}{10}$	$\frac{3}{4}(2) = \frac{3}{2}$	$\frac{7}{10} + \frac{3}{2} = \frac{11}{5}$

The states after the three updates are $(2, \frac{14}{5}, \frac{11}{5})$, or $(2, 2.8, 2.2)$. At position 2 the input is 10, but only one tenth of it enters the sum. The state therefore stays much closer to 2 than to 10. At position 3, the retention drops to one quarter, and the state moves from 2.8 toward the new input 2. Here the write coefficient is the complement of the retention because the exercise uses the chapter's restricted scalar system.

2. Connect the step to the write

In the zero-order-hold system, the input u stays constant during one interval of length Δ. The differential equation $\dot{x} = -x + u$ moves the state toward that held input. To see how much of the difference remains, let $d = x - u$. Since u is constant, $\dot{d} = -d$, so the difference at the end of the interval is $e^{-\Delta}$ times the starting difference.

Write the starting state as x_{start} and the ending state as x_{end}. The exact update has retention $a = e^{-\Delta}$ and write coefficient $1 - a$.

$$x_{end} = u + a(x_{start} - u) = ax_{start} + (1 - a)u.$$

Step Δ	Retention $e^{-\Delta}$	Write $1 - e^{-\Delta}$	Fraction of the starting distance still left
$\ln 2$	$e^{-\ln 2} = 1/2$	$1 - 1/2 = 1/2$	$1/2$
$\ln 4$	$e^{-\ln 4} = 1/4$	$1 - 1/4 = 3/4$	$1/4$

The distance still left is $a|x_{start} - u|$. Equivalently, the change in state is $(1 - a)(u - x_{start})$, which moves the state along the line from its starting value to the input. Since the starting state and input differ, **the step $\ln 4$ moves the state farther toward the input**. It covers three quarters of the original distance, while $\ln 2$ covers half. For example, from state 2 with input 10, the two steps give $2 + \frac{1}{2}(10 - 2) = 6$ and $2 + \frac{3}{4}(10 - 2) = 8$, respectively. The distance argument works in both directions, including when the input lies below the starting state.

3. Respect time order

Let $f(x) = 0.8x + 2$ be the first update and $g(x) = 0.5x - 1$ the second. An **affine map** multiplies by a slope and adds an offset. To combine two such updates, substitute the output of the first into the second. In the given time order this gives

$$g(f(x)) = 0.5(0.8x + 2) - 1 = 0.4x + 1 - 1 = 0.4x.$$

At $x = 3$, the first step gives $f(3) = 2.4 + 2 = 4.4$ and the second gives $g(4.4) = 2.2 - 1 = 1.2$. The combined map gives the same result, $0.4(3) = 1.2$.

Reverse the order by applying g first. The substitution is now

$$f(g(x)) = 0.8(0.5x - 1) + 2 = 0.4x - 0.8 + 2 = 0.4x + 1.2.$$

At $x = 3$, this order gives $g(3) = 1.5 - 1 = 0.5$ and then $f(0.5) = 0.4 + 2 = 2.4$. The combined map again agrees, $0.4(3) + 1.2 = 2.4$.

The slopes agree because the two scalar multipliers have the same product in either order. The offsets differ because the later step multiplies the offset from the earlier step. In the original order, the first offset 2 becomes $0.5(2) = 1$ before the second offset -1 is added. In the reversed order, the offset -1 becomes $0.8(-1) = -0.8$ before 2 is added. Grouping steps into a single map is valid as long as the updates keep their time order.

4. Follow one input to a read

Row 3 of M tells us how the three inputs contribute to the scalar output y_3. Follow one unit of input at a time. Its write column $\mathbf{b}_s$ has shape 2×1, so the write is a vector with one entry for each state coordinate. Each later retention multiplies both coordinates by the same scalar. Finally, the 1×2 read row $\mathbf{c}_3$ takes a dot product with that carried vector to produce one number.

The write at position s enters the state after the old state has been multiplied by a_s. It therefore passes through the transitions at positions $s + 1$ through 3. The recurrence makes the three paths explicit when the initial state is zero.

$$\begin{aligned} \mathbf{x}_1 &= \mathbf{b}_1 u_1, \\ \mathbf{x}_2 &= a_2 \mathbf{b}_1 u_1 + \mathbf{b}_2 u_2, \\ \mathbf{x}_3 &= a_3 a_2 \mathbf{b}_1 u_1 + a_3 \mathbf{b}_2 u_2 + \mathbf{b}_3 u_3. \end{aligned}$$

Here $a_1 = 0.6$ acts on the initial state, so it does not appear in a coefficient for any of the three inputs. The current write $\mathbf{b}_3 u_3$ is read at the same position where it is added. It crosses no transitions, so its **empty product** of retentions equals 1.

Input position s	Transitions before read at 3	Retention product	Dot product $\mathbf{c}_3 \mathbf{b}_s$	Entry $M_{3,s}$
1	$1 \to 2$ and $2 \to 3$	$a_2 a_3 = 0.5(0.8) = 0.4$	$2(1) - 1(2) = 0$	$0.4(0) = 0$
2	$2 \to 3$	$a_3 = 0.8$	$2(0) - 1(1) = -1$	$0.8(-1) = -0.8$
3	None	1	$2(1) - 1(0) = 2$	$1(2) = 2$

Thus **row 3 is** $(0, -0.8, 2)$, in input order, and $y_3 = 0u_1 - 0.8u_2 + 2u_3$. For input 1, the carried column per unit of input is $(0.4, 0.8)^\top$, which is nonzero. Its contribution to this particular read is zero because the read takes twice the first coordinate and subtracts the second. Input 2 reaches the read only through its second coordinate, which explains its negative coefficient. These entries are signed coefficients in a sequence matrix.

10. Design two memory tests (optional)

Imagine a delivery log in which a shipment can move more than once. One task asks for its location after the most recent update. The other asks where that same shipment started. Give both queries the same history up to the question. For example, consider these reports in order.

Report	Event
1	Delivery D17 is assigned to North.
2	Delivery D42 is assigned to West.
3	Delivery D17 moves to East.
4	Delivery D17 moves to South.
5	Delivery D42 moves to East.

For D17, the latest location is **South** and the first location is **North**. The report after its last update makes it necessary to identify both the correct delivery and the correct event. Always include cases with different first and last locations. If the two answers match, returning only the latest location could give the right answer on the first-location task without retaining the early fact.

Build synthetic logs with eight possible location labels of the same token length. Choose the first and last locations as a uniformly sampled ordered pair of distinct labels. This gives each location the same marginal probability of being the correct answer in either task. Randomize the delivery IDs and intermediate assignments. Keep the location labels from unrelated deliveries independent of the target's answer. Ask questions in a fixed format that names the target delivery and either *first* or *latest*.

Use two controlled sets of logs to separate different reasons for a change in accuracy.

Delay. Use 8, 32, 128, or 512 reports. Place the target's first assignment in report 2, one intermediate update near the middle, and its last update in report $R - 2$ when the log has R reports. Fill the remaining slots with unrelated status reports that contain no location labels. Keep the number of target updates, the distribution of answer labels, and the distance from the latest assignment to the question fixed.

Competing updates. At each length, compare zero, one, and two unrelated delivery updates that mention plausible locations. Add higher counts for longer logs. Replace neutral reports with the unrelated updates, placing them before, between, and after the target's updates when those slots are available. In a separate comparison at the same length, replace some neutral reports between the first and last assignments with more updates for the target. Keep the tokenized length and the positions of the first assignment, last assignment, and question fixed.

These comparisons distinguish a longer wait to recover the first fact, interference from other deliveries, and overwriting by updates for the target itself. Use report templates that tokenize to the same length in the competing-updates comparisons. Verify the actual token counts and evidence positions after tokenization. Reserve whole histories for the test split. The two questions and all variants of one history stay in the same split. Run the same prompts through each system with the same tokenizer and deterministic answer rule. For instance, require a single location label and count any unrecognized answer as incorrect.

Report exact-match accuracy separately for the two questions at each sequence length and interference level. Also report joint accuracy on each pair, which requires both answers to be correct, and count how often the first-location answer incorrectly matches the latest one. A model that gets the latest location right and gives that location again on the first-location prompt answers the update question correctly but fails to retrieve the original fact on this probe. Uniform guessing across eight labels has expected accuracy $1/8$ for either question.

Measure persistent inference memory immediately after consuming the history and before answering. Use batch size one and float32 for both state and cache arrays throughout this test. Sum the number of stored elements times their dtype size for each live array across all layers. For a selective state space layer with d feature channels and N state coordinates per channel, a state of shape $[1, d, N]$ uses $4dN$ bytes. Count any additional carried history, such as a short convolution buffer, separately and add it to the total. For an attention layer holding T positions with H_{KV} stored heads and key and value widths d_k and d_v, the key and value payload is

$$4TH_{KV}(d_k + d_v) \text{ bytes.}$$

Report both live bytes and allocated capacity if a system reserves extra cache slots. Keep model parameters and temporary workspace separate from persistent history. Plot or tabulate both task accuracies and persistent bytes at each length. This lets the reader see whether a state that keeps its size as the log grows still preserves the particular information each question requires.